

Security Audit

Teneo (Protocol)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviour	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	29
Conclusion	30
Our Methodology	31
Disclaimers	33
About Hashlock	34

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Teneo team partnered with Hashlock to conduct a security audit of their Teneo network smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts were secure.

Project Context

Teneo is an ecosystem built on the Ethereum Network that is part of a broader initiative to transform real-world assets into decentralized networks.

Users can purchase tokens from security pools, and stake and collect rewards for interacting with the protocol. This protocol acts as a one-stop shop for tokenizing company real-world assets (RWA) enabling users to transform machines into public investment opportunities.

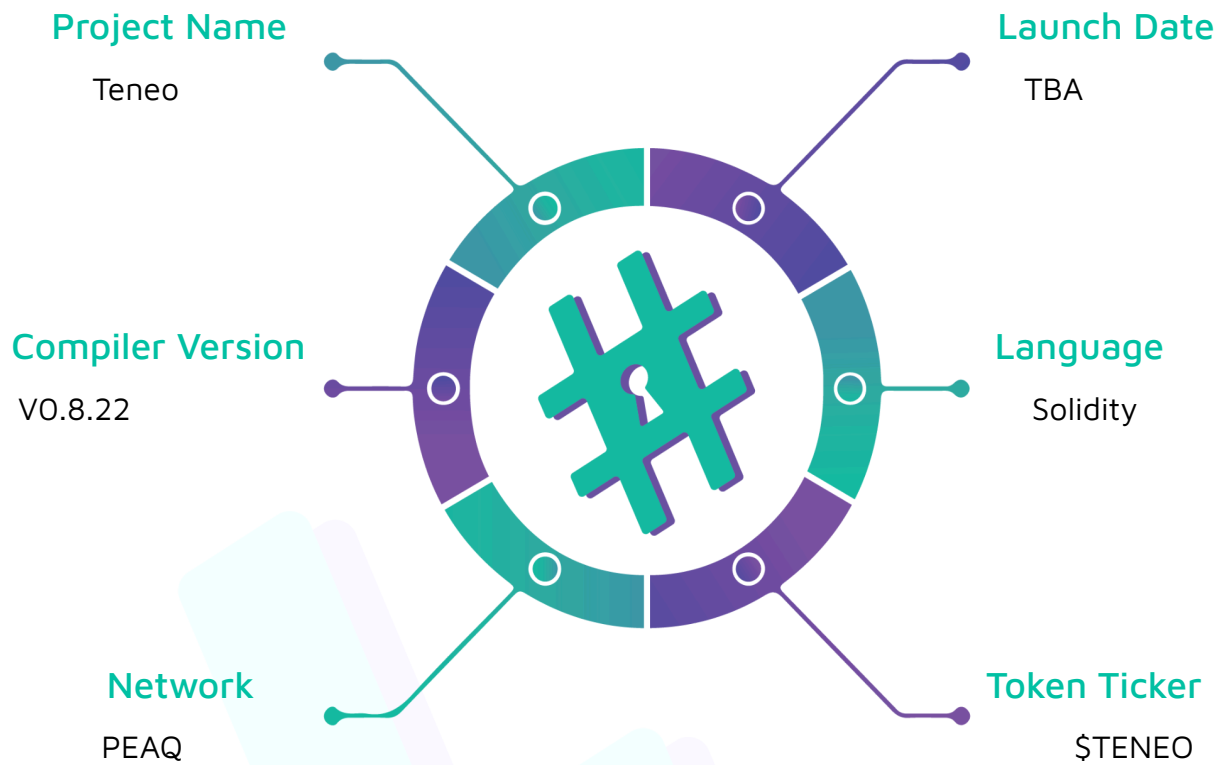
Project Name: Teneo

Compiler Version: ==0.8.22

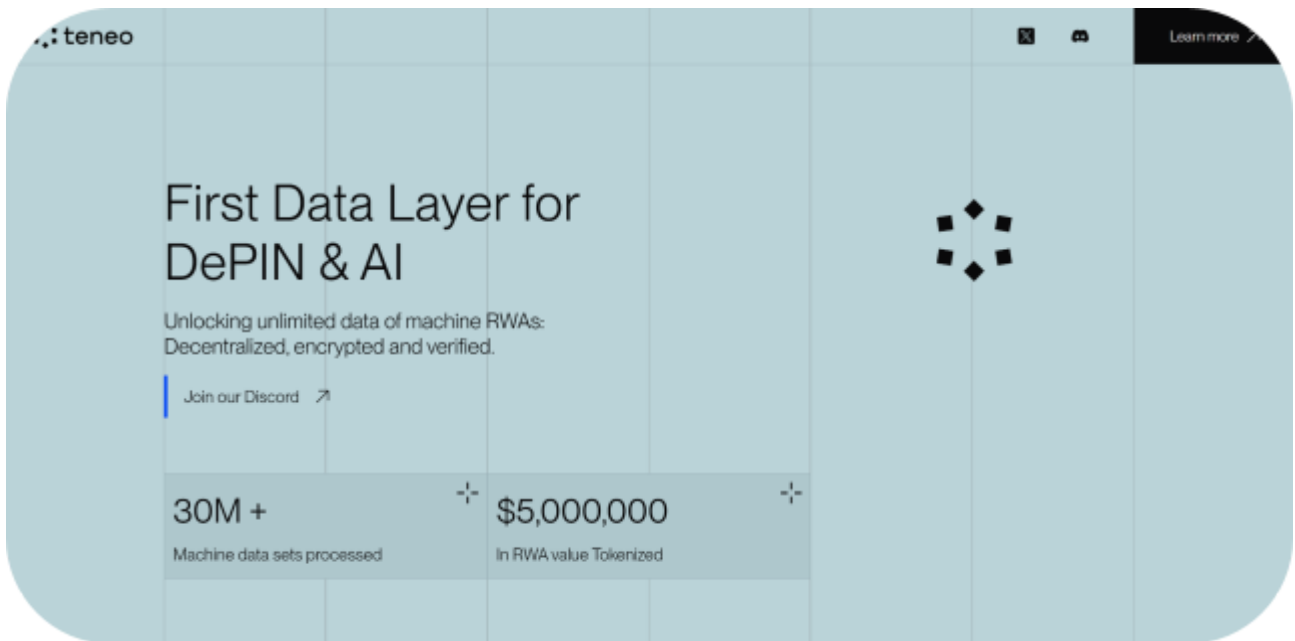
Website: <https://teneo.pro/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Teneo project, the scope of work included a comprehensive review of the project listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Teneo Smart Contracts
Platform	Ethereum / Solidity
Audit Date	June, 2024
TeneoRewardProtocol.sol	dcbd4151450c349198c4f32be55c087c
EnterpriseSecurityToken.sol	ba997ed46a992dfb5e41d662112580c1
TeneoToken.sol	678110ef49d5a7008c0876164ecffb1b
EnterpriseSecurityPoolFactory.sol	4ea50df7878bff7d16a5119255028272
MultiSigContract.sol	736abfc40fc48e49eea394b2a8c17c12
Utils.sol	3e212944567ae46358d0870f06bb3534
ERC1404/MessagedERC1404.sol	a8b293e697c1f5eb3afb782431b07045
ERC1404/MessagesAndCodes.sol	d2de1f6b40c0593a714eddbbd31833d7
ERC1404/SimpleRestrictedToken.sol	dc9ebc56cbf1e5130283f547758485ee

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

1 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviour

Claimed Behaviour	Actual Behaviour
Eloop.network.blockchain - Allows users to: - Stake tokens - Trade Security Tokens - Collect rewards - Allows admins to: - Modify permissions - Maintain the contract's day-to-day operations - Emergency withdraw value from the contracts in the event of a security incident	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts for the Teneo project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Teneo projects smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments and purpose of the protocol are all helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects. Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] TeneoRewardProtocol#transferFrom - `transferFrom` Does Not Spend From Allowance Which Can Allow Malicious Users to Steal Vault Tokens From Any User

Description

There exists an overridden function `transferFrom` which will allow users to transfer vault tokens between addresses. Whilst the relative permission bits are required to be set, tokens are not spent from allowance which can result in the theft of tokens.

Vulnerability Details

The `transferFrom` function will check with the utils contract if the relative permission bits have been set. The internal `_transfer(from, to, value)` function is then used to transfer value from user to user as can be seen from the implementation below:

```
/**
 * @dev Transfer function
 */
function transferFrom(
    address from,
    address to,
    uint256 value
) override public returns(bool success){
    if (utils.checkTransfer(address(this), from, to, value) != 0) {
        revert Errors.TransferRestricted();
    }
}
```

```

        _transfer(from, to, value);

        ===== SNIP =====

        return true;
    }

```

If a malicious user has `0x2` (receive) permission bit set, because the function does not first check allowance, they will be able to steal from anyone with `0x1` (send) bit set.

Proof of Concept

An example of a test that simulates this attack by stealing $20_000 * 1e18$ worth of security tokens from other users:

```

function testTransferFromAbuse() public {

    securityToken.debugMint(alice, 10_000 ether);

    securityToken.debugMint(bob, 10_000 ether);

    utils.setPermission(address(rewards), alice, 0x1);

    utils.setPermission(address(rewards), bob, 0x1);

    utils.setPermission(address(rewards), eve, 0x2);

    utils.setLocked(address(rewards), false);


    vm.startPrank(address(alice));

    securityToken.approve(address(rewards), type(uint256).max);

    rewards.stake(10_000 ether);

    vm.stopPrank();


    vm.startPrank(address(bob));

    securityToken.approve(address(rewards), type(uint256).max);

    rewards.stake(10_000 ether);

    vm.stopPrank();
}

```

```
vm.startPrank(eve);

rewards.transferFrom(alice, eve, 10_000 ether);

rewards.transferFrom(bob, eve, 10_000 ether);

vm.stopPrank();

assertEq(rewards.balanceOf(address(eve)), 20_000 ether);

}
```

Impact

Malicious users will be able to steal tokens from any vault token holder and will in turn allow them to steal their staked tokens.

Recommendation

It's recommended that the `transferFrom` function implements `_spendAllowance()` before triggering `_transfer`.

Status

Resolved

[H-02] TeneoRewardProtocol#distributeRewards - Whale Users Can Sandwich `distributeRewards` to Steal Rewards From Users Who Have Been Staked In The Protocol For Longer

Description

Contract administrators have a responsibility to maintain the rewards in the contract for users to collect when they have staked into the protocol. `distributeRewards` can be front-run by a user who has a substantial amount of tokens to receive more tokens than they are entitled to. The attackers are free to withdraw as soon as the rewards have been distributed.

Vulnerability Details

The `distributeRewards` function will pull tokens from the admin and into the contract. Following this, `magnifiedRewardPerShare` is calculated and the `lastRewardTime` is recorded:

```
function distributeRewards(uint256 amount)
    external
    adminOrBotCheck
{
    if (totalSupply() == 0) revert Errors.TotalSupplyZero();
    if (amount == 0) revert Errors.AmountZero();

    rewardToken.transferFrom(msg.sender, address(this), amount);

    magnifiedRewardPerShare += (amount * MAGNITUDE) / totalSupply();

    lastRewardTime = block.timestamp;

    emit RewardsDistributed(msg.sender, amount);
}
```

`withdrawReward` will then get the balance of the user in question and distribute rewards accordingly. Because the rewards distribution architecture does not consider the amount AND the duration of the staked user, a malicious user can exploit this by monitoring the mempool, observing a call to `distributeRewards` and execute a sandwich attack in order to get more rewards than other more entitled users.

Proof of Concept

Below is an example scenario where Alice and Bob stake 10_000 tokens each. Eve (a whale) frontruns the `distributeRewards` function and is able to collect half the rewards pool even though Alice and Bob have been staked much longer and are therefore entitled to more rewards:

```
function testWhaleStealsRewards() public {
```

```
// Setup Scenario

securityToken.debugMint(alice, 10_000 ether);

securityToken.debugMint(bob, 10_000 ether);

securityToken.debugMint(eve, 20_000 ether);

rewardToken.debugMint(address(this), 1_000 ether);

rewardToken.approve(address(rewards), type(uint256).max);

// Alice stakes into the protocol

vm.startPrank(address(alice));

securityToken.approve(address(rewards), type(uint256).max);

rewards.stake(10_000 ether);

vm.stopPrank();

// Bob stakes into the protocol

vm.startPrank(address(bob));

securityToken.approve(address(rewards), type(uint256).max);

rewards.stake(10_000 ether);

vm.stopPrank();

// Alice and Bob are staked for 1 week each

vm.warp(block.timestamp + 1 weeks);

// Eve frontruns a call to distribute rewards representing half the pool.

vm.startPrank(address(eve));

securityToken.approve(address(rewards), type(uint256).max);

rewards.stake(20_000 ether);

vm.stopPrank();

// Distribute rewards is called

rewards.distributeRewards(1_000 ether);
```



```
// Eve unstakes her tokens and claims her rewards immediately after

vm.prank(eve);

rewards.unstake(20_000 ether);

vm.prank(eve);

rewards.withdrawReward();


vm.prank(alice);

rewards.withdrawReward();


vm.prank(bob);

rewards.withdrawReward();


// Assert that the attacker has received more rewards than legitimate users

assertEq(rewardToken.balanceOf(eve), 4999999999999999999);

assertEq(rewardToken.balanceOf(alice), 2499999999999999999);

assertEq(rewardToken.balanceOf(bob), 2499999999999999999);

}
```

Impact

In the worst case scenario, a user with a substantial amount of capital can front run the call to `distributeRewards` by staking tokens, then back-run `distributeRewards` by collecting their rewards and withdrawing from the protocol. This was rated a High in severity because a user who has had zero contribution to the protocol can collect more rewards than loyal users who have been staking long term thus, stealing value from legitimate users.

Recommendation

It's recommended that the rewards functionality of this contract is refactored to either consider the staking duration of a user through an epoch system or force them to be staked for at least a minimum period of time before they are eligible for rewards.

Status

Resolved

Medium

[M-01] EnterpriseSecurityPool#buy - `buy` Does Not Employ SafeERC20 Which Can Result In Reverted Transactions For Popular Tokens Who Do Not Follow The ERC20 Standard

Description

The `buy` function uses a normal `transferFrom` in order to transfer value to and from the contract however, because some tokens such as USDT do not follow the ERC20 standard, may result in a revert due to a different `transferFrom` interface.

Vulnerability Details

In the implementation for the `buy` function below, it can be seen that a standard `transferFrom` function call is used:

```
function buy(uint256 amount) public nonReentrant whenNotPaused {  
    if (amount == 0 || tokenPrice == 0)  
        revert Errors.InvalidAmountOrPrice();  
    if (securityToken.balanceOf(address(this)) < amount)  
        revert Errors.InsufficientSecurityTokens();  
  
    uint256 transferAmount = tokenPrice * amount;
```

```

uint256 feeAmount = (transferAmount * utils.TeneoFee()) /

    utils.FEE_DENOMINATOR();

paymentToken.transferFrom(msg.sender, utils.feeReceiver(), feeAmount);

paymentToken.transferFrom(msg.sender, address(this), transferAmount);

securityToken.transfer(msg.sender, amount);

emit Buy(msg.sender, amount);

}

```

Because the standard ERC20 interface returns a bool, but USDT does not return anything these function signatures differ which can result in a revert due to an unknown function signature.

Proof of Concept

The proof of concept below demonstrates a buy order for security tokens using USDT forked from the mainnet but results in a revert:

```

function testUSDReverts() public {

    deal(address(usdt), alice, 1_000 * 1e6);

    vm.startPrank(alice);

    usdt.approve(address(pool), type(uint256).max);

    vm.expectRevert();

    pool.buy(1_000 * 1e6);

    vm.stopPrank();

}

```

Impact

Popular nonstandard ERC20 tokens such as USDT may not be used to purchase security tokens which can result in a loss of revenue for the protocol.

Recommendation

It's recommended that ERC20 transfers use the SafeERC20 library used by OpenZeppelin which uses a low-level call to ensure that tokens are safely transferred between addresses regardless of their compliance with the ERC20 standard.

Status

Resolved

[M-02] MultiSigContract#removeOwner - Missing Validation Checks in removeOwner Function of MultiSigContract

Description

The removeOwner function in the MultiSigContract lacks critical validation checks to prevent the removal of the msg.sender and the mainOwner. This oversight could lead to scenarios where the msg.sender inadvertently removes themselves or attempts to remove the mainOwner from the list of owners, potentially compromising the security and operational integrity of the contract.

Impact

Allowing msg.sender to remove themselves could unintentionally lock out administrative access if not properly managed, especially in scenarios where msg.sender is the last or a critical owner.

The ability to attempt the removal of the mainOwner contradicts the typical design pattern of having a non-removable, primary authority in multi-signature contracts, potentially leading to unintended consequences.

Recommendation

Implement two require statements at the beginning of the removeOwner function to validate that:

- The ownerToRemove is not the same as msg.sender.
- The ownerToRemove is not the mainOwner.

These checks ensure that the `msg.sender` cannot remove themselves or the `mainOwner`, preserving the administrative integrity and security of the contract.

```
function removeOwner(address ownerToRemove)

    public

    onlyWallet

    ownerExists(ownerToRemove)
{
    require(ownerToRemove != msg.sender, "removeOwner: cannot remove self");
    require(ownerToRemove != mainOwner, "removeOwner: Main Owner cannot be removed");

    isOwnerMap[ownerToRemove] = false;

    for (uint256 i = 0; i < owners.length; i++) {
        if (owners[i] == ownerToRemove) {
            owners[i] = owners[owners.length - 1];
            break;
        }
    }

    owners.pop();

    if (minSignatures > owners.length) changeMinConfirmation(owners.length);

    emit OwnerRemoved(ownerToRemove);
}
```

Status

Resolved

Low

[L-01] TeneoRewardProtocol#unstake - Unnecessary check of the user's balance

Description

The `unstake` function performs a check that is not necessary because it is already checked internally.

Vulnerability Details

The check of the user's balance is not necessary as it is internally checked by the burn function.

```
function unstake(uint256 amount) external whenNotPaused {  
  
    if (amount == 0) revert Errors.AmountZero();  
  
    if (balanceOf(msg.sender) < amount) revert Errors.InsufficientBalance(); //  
    @audit unnecessary check  
  
  
    // Burn the staked tokens to reduce the user's balance  
  
    burn(msg.sender, amount);  
  
  
    // Transfer the unstaked tokens back to the user  
  
    securityToken.transfer(msg.sender, amount);  
  
  
    emit Unstake(msg.sender, amount);  
  
}
```

Impact

A not needed operation is executed.

Recommendation

Remove the check

Status

Resolved

[L-02] Utils#setMultiplePermissions - Unbounded loop may run out of gas

Description

The `setMultiplePermissions` function may run out of gas if the transaction becomes too expensive.

Vulnerability Details

May run out of gas if the transaction becomes too expensive.

```
function setMultiplePermissions(AccessGroup[] calldata accessGroups)

    public adminOrBotCheck

{

    for (uint256 i = 0; i < accessGroups.length; i++) {

        AccessGroup calldata group = accessGroups[i];

        for (uint256 j = 0; j < group.accesses.length; j++) {

            UserAccess calldata access = group.accesses[j];

            for (uint256 k = 0; k < access.users.length; k++) {

                for (uint256 l = 0; l < access.accessIndices.length; l++) {

                    uint256 accessIndex = access.accessIndices[l];

                    address poolStaking = group.poolsStakings[accessIndex];

                    address user = access.users[k];

                    uint8 permission = access.permission;

                    participants[poolStaking][user] = permission;

                    emit PermissionSet(poolStaking, user, permission);
```

```

    }
  }
}
}
}

```

Impact

A scenario where permissions can not be set due to running out of gas can take place.

Recommendation

Create an auxiliary function to set the permissions 1 by 1 in case executing setMultiplePermissions becomes impossible due to gas consumption.

Status

Acknowledged

[L-03] TeneoRewardProtocol#distributeRewards - Protocol can become insolvent and lead to users not receiving rewards.

Description

If neither admin nor bot sends rewardToken to TeneoRewardProtocol then it can become insolvent.

Vulnerability Details

The function distributeRewards that implements adminOrBotCheck is used to transfer rewardToken to the contract to be used for rewarding the users. If no one calls this function and transfers the rewards then users will not be able to claim them.

```

/**

 * @dev Distributes rewards to stakers

 * @param amount The amount of rewards to distribute

```



```

    * Can only be called by an admin role

    */

function distributeRewards(uint256 amount)

    external

    adminOrBotCheck

{ // @audit protocol is insolvent if admin/bot decides to not call this function

    if (totalSupply() == 0) revert Errors.TotalSupplyZero();

    if (amount == 0) revert Errors.AmountZero();

    rewardToken.transferFrom(msg.sender, address(this), amount);

    magnifiedRewardPerShare += (amount * MAGNITUDE) / totalSupply();

    lastRewardTime = block.timestamp;

    emit RewardsDistributed(msg.sender, amount);

}

```

Proof of Concept

Below is a unit test that could be included under `TeneoRewardProtocol.test.ts` that shows that if no one calls `distributeRewards` function then the users are withdrawing 0 rewards:

```

it("should distribute rewards and allow users to withdraw their rewards without
owner calling distributeRewards", async function() {

    const initialUserBalance = await rewardToken.balanceOf(user.address);

    await rewardProtocol.connect(user).withdrawReward();

    const finalUserBalance = await rewardToken.balanceOf(user.address);

    const cumulateUserReward = await
rewardProtocol.connect(user).accumulativeRewardOf(user.address)

```

```
        console.log("cumulateUserReward", initialUserBalance, cumulateUserReward,
finalUserBalance)

        expect(finalUserBalance.sub(initialUserBalance)).to.be.equal(0);

    });
```

Impact

Under the described scenario users will not be able to withdraw rewards.

Recommendation

If it is desired that this process becomes more decentralised so that the users are not dependent on the admin or bot calling distributeRewards to allow them to withdraw rewards then the mechanism should be changed.

Status

Acknowledged

Gas

[G-01] MultiSigContract#removeOwner - Inefficient Multiple Reads of Storage Variable in removeOwner Function

Description

The function removeOwner in the MultiSignatureContract reads the storage variable owners multiple times within the same function. This practice is inefficient and can lead to increased gas costs due to the high cost associated with storage operations in Ethereum smart contracts. Specifically, the loop iterates through the owners' array to find and remove an owner, which involves multiple reads of the owners' storage variable.

Recommendation

To optimize gas usage and reduce costs, it is recommended to read the owners' storage variable once at the beginning of the function and assign it to a local memory variable. Subsequent operations should use this memory variable instead of directly accessing the storage variable multiple times. This approach minimizes the number of expensive storage reads and leverages the cheaper memory reads for operations within the function.

Storage operations are over 100x more costly than memory operations, making it crucial to minimize storage reads and writes where possible 4.

Utilizing memory variables for intermediate calculations or repeated reads can significantly reduce gas costs 4.

```
function removeOwner(address ownerToRemove)

    public

    onlyWallet

    ownerExists(ownerToRemove)
{
    address[] memory tempOwners = owners; // Read once and use a local memory variable
    isOwnerMap[ownerToRemove] = false;

    for (uint256 i = 0; i < tempOwners.length; i++) {
        if (tempOwners[i] == ownerToRemove) {
            tempOwners[i] = tempOwners[tempOwners.length - 1];
            break;
        }
    }

    owners = tempOwners; // Assign the modified array back to the storage variable

    owners.pop();
}
```

```
if (minSignatures > owners.length) changeMinConfirmation(owners.length);  
  
emit OwnerRemoved(ownerToRemove);  
  
}
```

Status

Acknowledged

Centralisation

The Teneo project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility for the contracts to run smoothly during their day-to-day operations.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Teneo project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd